Quiz 8: Study guide

DSAN-6600 Deep Learning, Fall 2026

Quiz 8 is Thursday Oct 22, at the end of class. About 15 minutes, 25 points.

Closed-book. No notes of any kind, and no AI. A basic calculator is allowed. A formula sheet is handed out with the quiz, so nothing needs memorizing.

⚠ This guide replaces the one posted on Oct 16, and the scope changed

The earlier guide described a sequence models quiz: padding, masking, `collate_fn`, bucketing, 1D convolution, recurrence, teacher forcing, backpropagation through time and gating.
The Oct 15 lecture did not reach any of that. It spent 95 of its 103 minutes finishing detection and segmentation, and reached sequences only in the last 8 minutes, stopping at embeddings. So the quiz covers what the lecture actually covered, and everything in that list moves to Quiz 9.
The quiz has been made easier because of this. Part A is now 8 true-or-false items instead of 6, every piece of computer vision arithmetic has been removed, and the only calculation left is the embedding one. The computer vision questions are answerable by somebody who sat through the lecture; none of them need a study guide.

❗ Format change: there is no coding section

Earlier quizzes ended with a "find the bugs in this script" exercise. That is gone, from this quiz and every later one. Part D is **short answer** instead: one or two sentences explaining why something works the way it does.
Reading code is still a skill this course uses constantly. Debugging syntax is not one it grades.

💡 If you only have ten minutes

Read these six things and nothing else.

1. **Anchors are fixed; the network predicts offsets from them.** (Part 2)
2. **Semantic segmentation labels pixels by class, instance segmentation labels them by object**, so only instance segmentation can count. (Part 7)
3. **A feature pyramid network helps small objects** because the deepest layers have thrown away the spatial detail small objects need. (Part 4)
4. **A ViT has no built-in locality**, so it needs more data than a CNN to learn what a CNN gets for free. (Part 8)
5. **An embedding is a row lookup, not a multiplication**, and its rows are learned. (Part 9)
6. **Subwords spell anything from pieces that are in the vocabulary**, which is why a 500-entry subword vocabulary beats a 10,000-word one. (Part 9)

That is 17 of the 25 points.

Scope

The Oct 15 lecture, and only as far as it got. That meeting spent its first 95 minutes finishing the computer vision deck, which had stopped the week before at non-maximum suppression, and reached sequence models only in the last 8 minutes.

On the quiz, from the computer vision half:

- region proposals, and two-stage detection as a shape
- ROI pooling, and what problem it solves
- **anchors and box offsets**: which part is fixed and which part is predicted
- **region proposal networks**, and why replacing selective search mattered
- **one-stage detection and YOLO**, including the output tensor
- one stage against two: what each trades away
- **feature pyramid networks**, and why small objects need them
- mean average precision, as a concept
- **semantic segmentation**, and the resolution problem
- **transposed convolution**, and the other two ways to upsample
- encoder and decoder shape, and **U-Net**
- **class imbalance** in segmentation, and why Dice exists
- **instance segmentation**, ROI Align, and how it differs from semantic
- **Vision Transformers**: patches, and what a ViT does not get for free
- **DETR**, as detection without anchors or NMS

And from the sequence half:

- what makes data sequential
- **tokenization** at character, word and subword level
- why subword tokenization won
- **one-hot encoding**, and why it is not the end of the story
- **embeddings**: a learned lookup table, one row per token

Not on the quiz. Padding, masking, `collate_fn`, bucketing, 1D convolution, recurrent networks, LSTM, gating, backpropagation through time, and anything about attention mechanics. None of it was reached. Lab 7 covered padding and masking; the lecture did not, so those move to Quiz 9.

Shape of the paper. Part A is 8 true-or-false items worth 1 each, Part B is 4 multiple choice worth 2 each, Part C is 3 points of fill-in-the-blank, and Part D is 3 short answers worth 2 each. Twenty-five points, about 15 minutes.

Part 1: Two-stage detection, finished

A two-stage detector proposes regions, then classifies and refines each one. Quiz 7 covered the first version, R-CNN, which used **selective search**: a non-trainable algorithm that groups similar pixels.

Everything after that is about removing the parts that were not learned.

version	what changed
R-CNN	selective search, then a CNN per proposal
Fast R-CNN	run the backbone **once**, crop features per proposal with **ROI pooling**
Faster R-CNN	replace selective search with a **region proposal network**

ROI pooling takes a proposal of any size on the feature map and produces a fixed-size grid, because the head after it is a fixed-size layer. ROI **Align** is the later fix: ROI pooling rounds proposal coordinates to feature-map cells, which misaligns the crop by up to half a cell, and alignment matters far more for masks than for boxes.

> **Likely question.** Which parts of a modern detector are trained? Everything, including the proposals. That is the point of the RPN.

Part 2: Anchors

An anchor is a fixed box shape, chosen in advance and tiled at every location on the feature map. Several scales and several aspect ratios, so a location with 3 scales and 3 ratios has 9 anchors.

The network does **not** predict anchors. For each one it predicts:

- an **objectness score**: is there an object here at all
- four **offsets** (t_x, t_y, t_w, t_h) that move and resize the anchor

$$x = x_a + t_x w_a, \quad y = y_a + t_y h_a, \quad w = w_a e^{t_w}, \quad h = h_a e^{t_h}$$

The exponential on width and height is so the predicted value can never be negative, and so the network predicts a *ratio* rather than an absolute size.

Why anchors exist at all: they are one answer to "how do you output a variable number of objects from a fixed-size network". Fix the number of slots in advance, then score and adjust each one. DETR is the other answer.

Part 3: One-stage detection and YOLO

A one-stage detector skips the proposal stage entirely. **YOLO runs the backbone once and reads boxes, objectness and class scores directly off a grid.**

The output tensor

This is the arithmetic worth being able to do cold. For an $S \times S$ grid, B boxes per cell, C classes:

$$\text{depth per cell} = \underbrace{B \times 5}_{\text{4 box + 1 objectness each}} + \underbrace{C}_{\text{class scores, shared across the cell}}$$

For $S = 7$, $B = 2$, $C = 20$: depth $= 2 \times 5 + 20 = 30$, so the output is $7 \times 7 \times 30$.

The common mistake is $2 \times (5 + 20) = 50$, which assumes each box has its own class scores. In the original YOLO they do not; the cell has one set.

One stage against two

	two-stage	one-stage
speed	slower, a head runs per proposal	faster, one pass
small objects	better	worse, historically
structure	propose, then refine	read it off the grid

two-stage	one-stage

Part 4: Feature pyramid networks

A deep backbone produces a sequence of feature maps that get **semantically stronger and spatially coarser**. The last one knows what things are but has thrown away where they are; a 16 pixel object may not survive to it at all.

An FPN takes the deep, semantically strong features and adds them back into the shallower, higher-resolution maps, then detects on several levels. So small objects are found on a map that still has the resolution to locate them, using features that are still good enough to classify them.

> **Likely question.** Why does an FPN help small objects? Because small objects need fine spatial detail, and the deepest layers have already discarded it.

Part 5: Semantic segmentation

Semantic segmentation assigns a class to every pixel. The output is the same height and width as the input, with one channel per class.

The resolution problem

A classification backbone deliberately shrinks its input, often by 32 times. For segmentation you need to get back to full resolution, which means upsampling.

Three ways to upsample

method	learned?	note
nearest neighbour or bilinear	no	cheap, safe, no artifacts
transposed convolution	yes	can produce checkerboard artifacts
upsample then ordinary convolution	yes	usually preferred now

A transposed convolution is not the inverse of a convolution. It is an ordinary convolution arranged to increase spatial size. It restores the *shape*, never the *values*. Its output size is

$$H_{\text{out}} = (H_{\text{in}} - 1)\, s - 2p + k$$

so with $k = 2$, $s = 2$, $p = 0$, an 8×8 map becomes 16×16. **Stride is what sets the upsampling factor.**

The **checkerboard artifact** happens when the kernel size is not divisible by the stride, so output pixels receive different numbers of contributions.

U-Net

An encoder that downsamples and a decoder that upsamples, with **skip connections** carrying the encoder's high-resolution feature maps straight across to the matching decoder level. Without them the decoder has to invent boundary detail from a coarse map, and segmentation is mostly about boundaries.

Part 6: Class imbalance and Dice

Here is the measurement that justifies everything in this section. A 128×128 image with one 16×16 object: the object is

$$\frac{16^2}{128^2} = 1.56\% \text{ of the pixels}$$

so a model that predicts **background everywhere** scores **98.4% pixel accuracy**. Cross entropy averages over pixels, so most of the loss is already satisfied and the object contributes almost nothing to the gradient.

Dice is computed from overlap instead:

$$\text{Dice} = \frac{2\,|P \cap G|}{|P| + |G|}$$

Predicting nothing gives $P = \emptyset$ and a Dice of 0, not 0.98. In practice the loss used is **cross entropy plus Dice**, because cross entropy gives well-behaved per-pixel gradients early in training and Dice stops the model settling for the majority class.

> **Likely question.** A model gets 98.4% pixel accuracy predicting nothing. What does that tell you? That accuracy and cross entropy are both dominated by the majority class, so neither one penalizes missing the object.

Part 7: Instance segmentation

Semantic segmentation labels pixels by class. Instance segmentation labels them by object.

The difference is sharpest when two objects of the same class touch. Semantically they are one connected region with no boundary between them; as instances they are two masks.

So the questions instance segmentation can answer and semantic cannot are anything involving **counting or identity**:

- how many cats are in this photograph
- which cat is closest to the camera
- is that one animal or two overlapping ones

This is exactly what Lab 6 measured on the course photograph: DeepLabV3 found a single "cat" region, Mask R-CNN found two separate cats.

Be able to write this answer in two sentences. It is Part D.

Part 8: Vision transformers, as a preview

A **ViT** cuts an image into fixed-size square patches, flattens each one, projects it to the model width, adds a learned position embedding, and runs a standard transformer encoder. A 224×224 image at patch size 16 gives $(224/16)^2 = 196$ patches of $16 \times 16 \times 3 = 768$ numbers each.

Nothing in that is image-specific except the patching.

The trade, which is the examinable part

	CNN	ViT
locality	built into the architecture	not present, must be learned
weight sharing	built in	not present
long-range relations	only after many layers	available from layer 1
data needed	less	more

On ImageNet-scale data alone a ViT underperforms a comparable CNN. Pretrained on far more data first, it matches or beats it. **So use a pretrained ViT or fine-tune one; training one from scratch on your own dataset is almost always wrong.**

DETR

A CNN backbone, a transformer encoder-decoder, and a fixed set of 100 learned **object queries**, each producing one class (including "no object") and one box. **No anchors and no NMS**, because both existed only to manage a variable number of outputs, and 100 fixed slots with a "nothing" option removes that need.

Part 9: Sequence data, as far as we got

What makes data sequential

Order carries meaning and the length is not fixed. Text, audio, video, DNA, any time series. A convolution assumes a fixed-size input and that the neighbours are the context, and a sentence satisfies neither.

Tokenization

Splitting the input into pieces, and a vocabulary mapping those pieces to integers.

level	sequence length	vocabulary	unseen input
character	longest	about 65	spells anything
word	shortest	never stops growing	becomes `<unk>`
subword	between	fixed	spells anything

Subword won because it gets the best of both: frequent words stay whole, rare words split into pieces that are themselves in the vocabulary, so the vocabulary is fixed and nothing is unspellable. `unhappier` becomes `un` + `happ` + `ier`, and a model that has never seen the word has seen the pieces.

Note the connection he drew in class: **patching an image is tokenizing it.** Same operation, different data.

One-hot encoding

Assign every token an integer, then represent it as a vector of zeros with a single 1 at that index. A sequence of s tokens over a vocabulary of n is an $s \times n$ matrix, almost entirely zeros.

Embeddings

Multiply that one-hot matrix by a weight matrix and you get a dense $s \times m$ matrix with m in the hundreds:

$$\mathbf{E} = \mathbf{O}\,\mathbf{W_E}$$

But nothing actually performs that multiplication. A one-hot row times the weight matrix selects a single row of the weight matrix and discards the rest, so a framework indexes instead. Building the sparse matrix and multiplying it would waste almost all of the memory and all of the arithmetic.

The rows are **parameters**. They start random and backpropagation shapes them, which is why `dog` and `cat` end up with similar vectors and `cat` and `bark` do not. One-hot can never do that: every token starts and stays exactly as far from every other.

`nn.Embedding(V, d)` maps $(N, T) \to (N, T, d)$ and holds $V \times d$ parameters. **Be able to do that both ways.**

> **Likely question.** Why does no framework perform the matrix multiplication? Because the product is a row lookup, and the matrix is almost all zeros. This is Part D.

How to prepare

1. **Be able to say what each piece is for in one sentence**: an anchor, an objectness score, ROI pooling, an FPN, a transposed convolution, a skip connection, Dice loss, a patch, an object query, a token, an embedding row.
2. **Do the two arithmetic items cold**: the YOLO per-cell count, which the question spells out for you, and an embedding's output shape and parameter count. There is no other calculation on this quiz.
3. **Practise two short answers out loud**: why an embedding is a lookup rather than a multiply, and what instance segmentation answers that semantic cannot.
4. **Know which part of each system is learned** and which is fixed. Anchors fixed, offsets learned. Selective search fixed, RPN learned. Positional patches fixed, position embedding learned.

The formula sheet carries IoU, the anchor offsets, the YOLO depth, both convolution size formulas, Dice, the patch count and the embedding shapes. It deliberately does not say what any of them are *for*. That part is the quiz.